

BranchMem: Boundary Conditions for Asynchronous Semantic Agent Memory Merging

Anonymous ACL submission

Abstract

Reconciling two branches of an LLM agent’s memory that diverged while disconnected differs from live multi-agent coordination and single-timeline freshness: no shared clock, no live negotiation. We introduce BRANCHMEM, a preregistered benchmark and harness to identify *when* such merging is reliable, not to claim it solved. Ancestor-aware LLM merging beats naive baselines by wide margins, but only when provenance carries a legible reliability signal: strip it and the LLM collapses to naive-baseline territory, while CONFIDENCERULEMERGE — a deterministic rule reading the same field — matches or beats it, the strongest baseline here. On real, branch-forked MemoryAgentBench content, where no such field exists, the ranking inverts: across 230 conflicts the LLM never guesses, correctly flagging the absent signal, which accuracy alone penalizes as error. We report abstention-aware metrics showing calibrated abstention is correct once a signal is absent. Cheap conflict detection is a separate, mixed result on a harder, balanced test set. We release the harness, preregistered plan, and every cached call.

1 Introduction

Language-model agents increasingly carry a persistent, natural-language *belief state* that is naturally replicated across devices, sessions, or replicas. When two replicas operate while disconnected — a phone assistant used offline while a desktop copy keeps learning — each accumulates its own updates; on reconnection, the two *branches* of memory must be merged into one.

This paper is about *when*, not *whether*, that merge can be trusted: our central finding is a boundary condition, not a method. LLM-mediated merging is reliable exactly when branch provenance carries a legible reliability signal, a deterministic rule reads it just as well, and, on real content where no such signal is engineered in,

the right behavior is to abstain rather than guess, which accuracy alone scores as a mistake.

The asynchronous partition-and-merge problem differs from three related lines of work. It is not *live multi-agent coordination*: systems such as S-Bus (Khan, 2026) abort or retry conflicting commits via optimistic concurrency control, presupposing agents share a substrate online — our branches were never online together, so there is nothing to abort, only two already-committed histories to reconcile. Nor is it *single-timeline temporal freshness*: on one ordered timeline a deterministic max(serial) rule beats asking the LLM to judge freshness (Reddy and Challaram, 2026), but disconnected branches share *no clock*; “later” on one branch’s timestamp says nothing about correctness relative to the other. Nor is it *structured, CRDT-style replication*: CRDTs (Shapiro et al., 2011) and convergent-log substrates (Sarkar, 2026) resolve divergence with a commutative merge function so any two replicas provably reconcile; free-form language has no such operator — whether “the user prefers window seats” and “prefers aisle seats” should merge, split, or flag is a semantic judgement, precisely where an LLM might help over naive alternatives (last-writer-wins, keep everything, drop a branch), if it buys anything over these cheap defaults at all.

Why abstention matters here. A wrong merge silently corrupts the agent’s belief, and every downstream decision inherits the error; we cannot abort, as coordination systems can, since branches already committed offline. When no signal distinguishes the correct value, forcing a commit is a coin flip dressed as an answer; flagging it trades a lower accuracy score for a lower chance of propagating a wrong belief — a tradeoff accuracy alone hides (Section 5.8).

We formalize this problem and build BRANCHMEM, a harness that forks a common-ancestor

memory, applies independent scripted updates to two branches, and scores how well merge strategies recover a construction-time ground truth, studying three questions: **RQ1**, does an explicit *three-way* LLM merge—ancestor plus both branches—beat naive last-writer-wins and concatenation? **RQ2**, can conflict *detection* be done cheaply (embedding similarity, a small NLI model) instead of with an LLM judge? **RQ3**, does reconciled memory beat discarding one branch entirely (the post-hoc analogue of an abort)?

Hypotheses, statistical tests, effect sizes, and falsification criteria were locked *before* the confirmatory run; every subsequent addition is disclosed as post-hoc and kept separate (ANALYSIS_PLAN_ADDENDUM.md), including results unfavorable to the headline claim (Sections 5.2, 5.7). Our contributions: (i) we formalize asynchronous partition-and-merge for semantic agent memory, distinct from live coordination, single-timeline freshness, and CRDT-style replication; (ii) we introduce BRANCHMEM, a reproducible benchmark and harness for forked semantic memories, every real model call cached and released; (iii) we show LLM-mediated merging beats naive baselines only when provenance carries a legible reliability signal (RQ1, RQ3); (iv) we show a deterministic, confidence-based rule (CONFIDENCERULEMERGE) is a strong baseline that matches or beats the LLM under the current construction (Section 5.2); (v) we add abstention-aware evaluation (Section 5.8) — commit rate, conditional accuracy, and expected utility — showing when FLAGGED_UNRESOLVED is safer than forcing a resolution; (vi) we evaluate conflict detectors on a balanced conflict/non-conflict set (Section 5.4), not only positive examples, giving RQ2 real precision, not just recall; and (vii) we report a real-content MemoryAgentBench extension (Section 5.7) showing the synthetic provenance signal does not automatically exist in realistic memory data — the boundary condition this paper is built around.

2 Related Work

Four further points of comparison — agent memory benchmarks (Shinn et al., 2023; Packer et al., 2023; Hu et al., 2025), live multi-agent coordination (S-Bus’s *prevention/abort* model, Khan, 2026), single-timeline temporal freshness (a deterministic “pick the maximum serial” recipe, Reddy and Challaram, 2026), and conflict detection via

embeddings/NLI (Longpre et al., 2021; Lewis et al., 2020; Reimers and Gurevych, 2019) — are each close enough to our setting to warrant full discussion; we give it in Appendix L rather than here, since the core distinctions are already established in Section 1, and the detector background supports only RQ2, not the paper’s central claim.

2.1 Structured replication, CRDTs, and local-first software

CRDTs (Shapiro et al., 2011) and operational transformation (Ellis and Gibbs, 1989) give provably convergent, commutative merge for structured data; convergent-log agent substrates (Sarkar, 2026) and local-first software (Kleppmann et al., 2019) build on the same primitives. Convergence needs a merge operator with lattice-like structure, which free-form language lacks; BRANCHMEM studies what to do without one, using an LLM as a best-effort reconciler rather than a guaranteed-convergent one.

2.2 Abstention and selective prediction

Selective classification (El-Yaniv and Wiener, 2010; Geifman and El-Yaniv, 2017) formalizes the coverage-risk tradeoff used in Section 5.8: a classifier that may abstain trades coverage against risk rather than being forced to one accuracy number. FLAGGED_UNRESOLVED is a selective-prediction abstention; accuracy alone cannot distinguish a calibrated abstention from a wrong guess.

3 Problem Formulation and Method

3.1 Branch, fork, and merge

We represent memory as a set of *facts*, each a canonicalized tuple (*entity*, *predicate*, *value*) with metadata (*source*, *confidence*, *timestamp*), where *source* $\in$ {user, observation, inference}. The pair $k = (\textit{entity}, \textit{predicate})$ is a *key*; the current value at a key is the fact with the latest branch-local timestamp for that key.

A *common ancestor* M_0 is forked into branches A and B , each initialized as a copy of M_0 . Each branch then applies an independent stream of scripted updates, superseding prior facts at a key while retaining full history. Every key touched by both branches yields a *conflict pair*. A merge strategy consumes (M_0, A, B) and returns a reconciled fact set in which each key resolves to KEPT (one branch’s value, or agreement), MERGED (a reconciled value), DROPPED (lost), or FLAGGED_UNRESOLVED (deferred to the user). Ground-truth conflict labels and correct values are

attached *by construction* at scenario-generation time, never inferred afterward.

3.2 Merge strategies

We compare four families, five strategies in total.

LastWriterWins (LWW). At each conflicting key, keep the fact with the later timestamp — the strategy the freshness recipe (Reddy and Chalam, 2026) would reduce to, but uninformative here since the branches have no shared clock.

NaiveConcat. Keep every current fact from both branches: never drops information, but never resolves a genuine contradiction either — both conflicting values survive, so downstream reads see an inconsistent store.

BranchDiscard. On *any* detected key collision, discard one branch’s entire post-fork update set (including non-conflicting additions) and keep the other — the post-hoc analogue of an abort. Two policies: `always_b` (always drop *B*) and `fewer_updates` (drop the branch with fewer updates); facts identical to the kept branch’s value are not counted as lost.

ThreeWayLLMMerge (proposed). Analogous to a three-way file merge, the LLM gets the common ancestor’s value for each conflicting key plus both branches’ current value, source, and confidence, and reconciles each with a one-sentence justification or flags it unresolvable. Non-conflicting keys resolve deterministically in code; only genuine collisions reach the model, batched into one call per merge. The system prompt instructs it to use source/confidence and ancestor context—not timestamps, since disconnected branches share no clock—and to flag rather than guess absent a distinguishing signal (full prompt in Appendix A; worked example in Appendix B).

3.3 Conflict detectors (RQ2)

Merge strategies identify candidate conflicts structurally (same key, differing current value), which is realistic for a canonicalized fact store. RQ2 asks a separate question: can we *detect* whether two natural-language values genuinely conflict without an LLM? We compare three detectors, treating the LLM judge as the reference standard, not a competitor.

EmbeddingConflictDetector. Since candidate pairs already share a key, low cosine similarity between the two *value* strings (sentence embeddings, `all-MiniLM-L6-v2`, Reimers and Gurevych, 2019) signals a real conflict, high similarity a paraphrase; a calibrated threshold decides.

NLIConflictDetector. A small NLI cross-encoder (`nli-deberta-v3-small`) scores the contradiction probability between the two facts phrased as short sentences; a calibrated threshold decides.

LLMJudgeConflictDetector. An LLM judges whether the two values conflict — the RQ2 reference standard. Both cheap detectors run locally at no API cost; the LLM judge does not.

3.4 Scenario generator and conflict taxonomy

The generator builds fully deterministic scenarios (given a seed) from a fixed pool of entities, *objective* predicates (with a fact-of-the-matter), and *subjective* predicates (without one). Each scenario forks a 4-fact ancestor and scripts three kinds of divergence:

Orthogonal. A key touched by exactly one branch (a new, non-contradictory addition). The correct behavior is to *preserve* it; this is where `BRANCHDISCARD` structurally loses information.

Resolvable conflict. Both branches update the same key with contradicting values, and there *is* a single correct reconciled value, tied to a *source-reliability signal*: one branch’s update is user-sourced with high confidence, the other inference-sourced with low confidence, and ground truth is the user-sourced value—with 15% noise, so a strategy cannot get a free 100% by blindly trusting the user source. This signal was not incidental; it resolves a construct-validity problem uncovered in piloting (Section 6).

Ambiguous conflict. Both branches update the same *subjective* key with different values and the *same* source/confidence—there is deliberately no distinguishing signal, so the only correct behavior is to `FLAG`, not guess. Ground truth is “unresolved.”

A separate parameter, `timestamp_correlates_with_correctness`, set to 0.5, controls only whether the correct branch also happens to carry the later timestamp—uninformative about correctness at 0.5, the realistic asynchronous case, so LWW is neither trivially right nor trivially wrong. This value and the source-reliability construction were preregistered before the locked run. Downstream questions are generated one per conflict pair and per orthogonal addition, scored against the generator’s ground truth by Python equality, never against any model’s opinion.

3.5 MemoryAgentBench extension

To test realistic content, we extend MemoryAgentBench’s Fact Consolidation task (Hu et al., 2025) with a branch-fork step. Since its published data exposes only a flattened context string, we built a 38-template regex parser recovering (*entity, predicate, value*) triples at 99.9% coverage over 18,337 context lines. Coverage alone does not establish correctness, so we additionally ran an *author self-audit* (not independent validation, explicitly labeled as such) on 50 of the 51,314 parsed triples: 0 errors (95% Clopper–Pearson CI [0, 0.071]). End-to-end validation against the live dataset surfaced two construction bugs (Section 6); after fixing them, LWW accuracy clustered near chance (49–56%) across 17,816 conflict pairs, as expected for an unbiased construction. Fact Consolidation is deterministic, so this extension yields only resolvable and orthogonal divergences; our *primary* experiment runs on the synthetic generator instead, where the resolvable/ambiguous mix is controllable, and the MAB extension is a validated secondary path (Section 5.7), not part of the locked analysis.

One difference matters for Section 5.7: the MAB extension does *not* construct a source-reliability signal — its UpdateSpecs inherit the dataset’s own provenance, so both branches default to `source="observation"`, `confidence=1.0`, no `user-vs.-inference` distinction. This is not an oversight to patch; real, reconstructed content lacks the clean annotation we hand-designed for the synthetic generator (Section 3.4), and we treat that absence as the realistic case worth measuring.

4 Experimental Setup

Preregistration. After a pilot phase used to fix bugs and calibrate thresholds, we committed an analysis plan specifying the hypotheses, statistical tests, minimum effect sizes, F1 tolerance, detector thresholds, sample size, and falsification criteria *before* the locked run. The locked run then executed exactly that plan with no parameter changes.

Model. All LLM-dependent components (THREWAYLLMMERGE, LLMJUDGE) use gpt-5.4-nano, the cheapest OpenAI model available to us. This was an explicit cost-minimization constraint, not a capability claim; whether effects hold with stronger models is future work.

Scenarios and seeds. The locked run uses $n=60$ scenarios (20 each at divergence spans 4.0, 10.0, 20.0), seed 2026, yielding 180 ground-truth conflict pairs (120 resolvable, 60 ambiguous). Each scenario has 1 entity, a 4-fact ancestor, two orthogonal additions per branch (Appendix F notes a documentation correction here), two resolvable conflicts, and one ambiguous conflict.

Detector calibration. Embedding and NLI thresholds were swept against the 60 ground-truth conflict pairs of the *pilot* set (held out from the locked set); precision was 1.0 across the entire sweep for both detectors—only false negatives. We locked embedding threshold 0.80 (start of its F1 plateau) and NLI threshold 0.20 (middle of its higher-F1 plateau), choosing plateau interiors for robustness to distribution shift (full sweep in Appendix E).

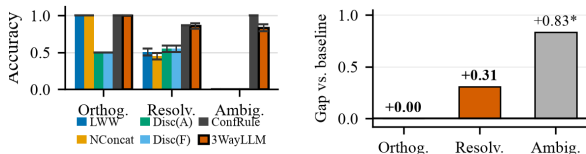
Statistics. For RQ1/RQ3 we use per-scenario paired accuracy differences, a paired Wilcoxon signed-rank test per comparison (chosen over a *t*-test since per-scenario accuracy is a bounded proportion over few questions), Holm–Bonferroni correction across the four confirmatory comparisons, and 2000-resample paired bootstrap 95% CIs. The minimum meaningful effect is 0.10 accuracy (preregistered from pilot variance; support requires both corrected significance and a CI lower bound above 0.10). For RQ2 we use a fixed 0.05 F1 tolerance band relative to the LLM judge—no significance test—matching the build spec’s example value.

Non-determinism. Pilot resampling showed a low flip rate ($1/15=6.7\%$ for the merge, $0/10$ for the judge), so the locked run calls each LLM-dependent component once per scenario/pair rather than majority-voting, a preregistered cost/variance tradeoff disclosed as a limitation.

Reproducibility. Every real model call across pilot and locked run (460 calls total; 111,403 input / 45,462 output tokens) is content-hash cached and committed as an auditable artifact; re-running costs nothing once the cache is populated.

Post-hoc robustness checks (not preregistered). We ran two additional checks after the locked analysis was complete, disclosed as exploratory rather than confirmatory: a model swap on a scenario subset, and a secondary run on real MemoryAgentBench content. Both are described where reported, Sections 5.6–5.7.

5 Results



(a) Accuracy by category, (b) Margin over best baseline.

Figure 1: Table 2 visualized, with CONFIDENCERULEMERGE (gray, post-hoc) promoted alongside the locked-run strategies. (a) Black outline = THREEWAYLLMMERGE; CONFIDENCERULEMERGE matches or beats it on resolvable and ambiguous. (b) Margin over best *naive* baseline: resolvable (+0.31) is genuine; ambiguous (+0.83, gray, *) is mechanically guaranteed.

5.1 RQ1: three-way merge vs. naive baselines (H1) — supported, but concentrated in one category

Three-way merge scores 0.936 mean downstream accuracy overall, versus 0.717 (LWW) and 0.700 (naive concat); both comparisons (Table 1) clear the 0.10 minimum effect by a wide margin. Taken at face value this looks like a broad win over naive baselines; Table 2 and Figure 1 show it is concentrated, and Section 5.2 shows it is not a win over the *strongest* baseline. All strategies tie at 1.000 on orthogonal questions, contributing nothing to the gap. On **resolvable** conflicts, three-way merge scores 0.858 versus 0.45–0.55 for the naive baselines: given the ancestor plus each branch’s source/confidence, the model identifies the more-reliable branch most of the time, while the naive strategies do little better than the $\approx 50\%$ chance rate implied by our timestamp-uncorrelated design. **This is the paper’s real, non-mechanical finding against naive baselines** — though Section 5.2 shows a deterministic rule reading the same signal scores 0.867 here, edging out the LLM. On **ambiguous** conflicts, three-way merge scores 0.833 versus 0.000 for every naive baseline: no naive strategy can emit FLAGGED_UNRESOLVED at all, so none could score above 0 regardless of reasoning quality — though again, CONFIDENCERULEMERGE matches this by flagging exact-confidence ties (Table 2). Section 5.7 shows the resolvable-conflict finding itself does not survive removing the hand-designed reliability signal — the single most important qualifier to this result. Divergence duration had little effect (0.914/0.957/0.936 at spans 4.0/10.0/20.0; per-span breakdown in Ap-

pendix C). **Reframing RQ1:** THREEWAYLLMMERGE beats naive baselines but not the strongest provenance-aware deterministic baseline, so the LLM’s potential value must be tested where raw text or ancestor context carries information a source/confidence field does not (Section 5.5).

5.2 Ablations: the strongest baseline is deterministic abstention, not LWW

Three post-hoc ablations on the same $n=60$ scenarios test whether THREEWAYLLMMERGE needs its full source/confidence/ancestor design. The first result reframes the whole comparison: naive last-writer-wins is *not* the strongest alternative to the LLM. CONFIDENCERULEMERGE makes no LLM call: keep the higher-confidence branch, flag on an exact tie. It scores 0.962 — *higher* than THREEWAYLLMMERGE’s 0.936 (paired diff -0.026 , 95% CI $[-0.040, -0.014]$, $p=0.002$): a single if-statement outperforms the LLM, since higher confidence is correct with probability $1 - \text{source_signal_noise} = 0.85$ by construction and ties (the ambiguous case) are flagged just as reliably by a rule as by judgment. TWOWAYLLMMERGE withholds the ancestor entirely; it scores 0.926, statistically indistinguishable from the full version ($p=0.29$) — ground truth here is tied to source/confidence, not ancestor context, so the ancestor is not yet load-bearing in this construction.

RAWTEXTLLMMERGE: withhold everything. TWOWAYLLMMERGE still hands the model a structured confidence field — exactly what CONFIDENCERULEMERGE reads directly. To isolate LLM reasoning about the raw text from reading that field, RAWTEXTLLMMERGE’s prompt gives *only* the entity, predicate, and each branch’s raw conflicting value — no ancestor, source, confidence, or timestamp. It scores 0.712 overall — close to the naive baselines (0.70–0.72) and far below THREEWAYLLMMERGE (paired diff $+0.224$, 95% CI $[0.200, 0.248]$, $p < 10^{-10}$). The category breakdown is stark: 1.000 orthogonal, 0.000 **resolvable** — *worse than every naive baseline* — and 0.983 ambiguous. With no structured signal, the model does not guess from content; it flags almost everything, exactly correct on ambiguous conflicts and exactly wrong, under our scoring rule, on resolvable ones — essentially none of RQ1’s resolvable-category win is attributable to the LLM reasoning about the raw text itself.

Hyp.	Comparison	Mean acc. diff	95% CI	Holm-adj. p	Supported?
H1	three-way vs. last-writer-wins	+0.219	[0.183, 0.252]	1.2×10^{-10}	Yes
H1	three-way vs. naive-concat	+0.236	[0.205, 0.267]	1.2×10^{-10}	Yes
H2	three-way vs. branch-discard (always_b)	+0.493	[0.460, 0.526]	3.7×10^{-11}	Yes
H2	three-way vs. branch-discard (fewer_updates)	+0.493	[0.460, 0.526]	3.7×10^{-11}	Yes

Table 1: Confirmatory RQ1/RQ3, locked set ($n=60$, Holm–Bonferroni corrected). All four clear the preregistered 0.10 effect — **decomposed in Table 2, qualified in Section 5.7.**

Strategy	Orthog.	Resolv.	Ambig.
last_writer_wins	1.000	0.508 $\pm$.359	0.000
naive_concat	1.000	0.450 $\pm$.325	0.000
branch_discard (always_b)	0.500	0.550 $\pm$.325	0.000
branch_discard (fewer_upd.)	0.500	0.550 $\pm$.325	0.000
CONFIDENCERULEMERGE [†] (no LLM)	1.000	0.867	1.000
three_way_llm	1.000	0.858 $\pm$.275	0.833 $\pm$.373

Table 2: Accuracy by category ($n=60$; SD in small type). Orthogonal ties; resolvable is the genuine gap, where CONFIDENCERULEMERGE edges out the LLM (0.867 vs. 0.858); ambiguous is flagged by both confidence-aware strategies. [†]Post-hoc addition, zero LLM cost, not part of Table 1. Visualized in Figure 1.

Strategy	Mean acc.
CONFIDENCERULEMERGE (no LLM)	0.962
THREEWAYLLMMERGE (proposed)	0.936
TWOWAYLLMMERGE (no ancestor)	0.926
RAWTEXTLLMMERGE (no metadata)	0.712

Table 3: Ablations, same $n=60$ scenarios; exploratory, post-hoc. RAWTEXTLLMMERGE added as a later post-hoc check.

Reading. CONFIDENCERULEMERGE also fails on real MemoryAgentBench content (Section 5.7) for the same reason — not evidence the LLM is useless, only that this benchmark’s current construction is well-approximated by reading one metadata field. RAWTEXTLLMMERGE sharpens this: without that field, the LLM’s own judgment of the raw values matches its real-content behavior, not its synthetic resolvable-category behavior. Crediting genuine semantic reasoning needs cases where confidence alone is insufficient *and* the raw text carries a resolvable signal, which the generator does not yet build — future work, not a reason to retract RQ1’s aggregate result.

5.3 RQ3: three-way merge vs. branch-discard (H2) — supported by a near-mechanical margin

Three-way merge beats both branch-discard policies by +0.493 (Table 1) — the largest effect in the study, and the one we flag as carrying the *least* evidentiary weight. Both policies scored identically (0.443 overall): single-entity scenarios with symmetric update counts leave nothing for fewer_updates to distinguish, and discard-

Detector	F1	Gap	≤ 0.05 ?
embedding ($t=0.80$)	0.957	0.029	Yes
NLI ($t=0.20$)	0.925	0.061	No
LLM judge (reference)	0.986	—	—

Table 4: RQ2: conflict-detection F1 (locked set, 180 pairs) vs. LLM-judge reference. Gap = $|F1_{\text{cheap}} - 0.986|$; tolerance 0.05.

ing a whole branch mechanically throws away its non-conflicting orthogonal additions, present in every scenario. Disclosed in advance, H2 support was expected almost by construction; we report it as **confirmation of a predicted-by-design failure mode**, not evidence on par with Section 5.1’s resolvable-conflict finding.

5.4 RQ2: cheap detectors vs. LLM judge (H3) — mixed

Table 4 gives the result: embedding F1 0.957 vs. judge 0.986 (gap 0.029, within the 0.05 tolerance, **supported**); NLI F1 0.925 (gap 0.061, **falsified**). This is a genuine *generalization gap*, not a fixed-threshold artifact: the judge’s own F1 moved from 0.966 on the pilot calibration set to 0.986 on the independently-generated, three-times-larger locked set, and bootstrapped 95% CIs on the gap itself (embedding 0.029 [0.006, 0.056]; NLI 0.061 [0.036, 0.089]) show this is consistent with ordinary sampling variation, not a systematic drift; NLI’s interval straddles 0.05, so **falsified** holds at the point estimate but not free of sampling uncertainty (full detail in Appendix E). Precision is 1.0 for every detector because every ConflictPair here is a genuine conflict by construction, so 1.0 reflects pair construction, not measured specificity (Section 5.4 adds real negative examples). Both cheap detectors’ errors are exclusively missed conflicts, and both run ≈ 5 – $10\times$ faster than the judge at no API cost.

Post-hoc: a balanced detector benchmark. Since the locked set above has no negative examples by construction (so precision is mechanically 1.0), we built a balanced, hand-authored 48-pair set spanning six categories and re-ran all three de-

tectors: the embedding detector’s locked-set precision turns out to be a construction artifact (it is the *weakest* on this harder set), while NLI reverses its “falsified” standing (best cost-free option here). Post-hoc, exploratory; does not change RQ2’s locked result. Full setup, metrics, and per-category breakdown in Appendix I.

5.5 Does raw text ever carry the resolving signal?

RAWTEXTLLMMERGE (Section 5.2) strips source/confidence, but every “resolvable” conflict there still ties ground truth *only* to that metadata. We built a disjoint-seed category, semantic_resolvable ($n=150$, seed 3001): identical source/confidence on both branches, with the ancestor holding a stated constraint at the *same* key as the conflict (e.g. ancestor “is vegetarian”; branch values “wants chicken” vs. “wants tofu”), so the answer is recoverable only from semantic compatibility with that constraint — a fair test of whether raw text or ancestor semantics can resolve a conflict on their own. Every confidence-independent strategy, *including* THREEWAYLLMMERGE which receives the constraint, scores *exactly* 0.000; naive baselines score 0.47–0.53 (chance-level, as designed). We read this as a genuine negative result: giving the model the necessary semantic content did not produce reliable semantic resolution. Full construction detail, a construct-validity bug we caught and fixed on the first run, and transcript analysis are in Appendix J.

5.6 Post-hoc robustness checks: a more capable model, and a resolvable-category replication

Two post-hoc robustness checks — a gpt-5.4-mini model swap on the same 20 scenarios, and a second, disjoint-seed 120-pair resolvable-category sample for more statistical power — both replicate RQ1’s direction and magnitude with no qualitative change (mini vs. nano paired diff. +0.014, 95% CI $[-0.014, 0.043]$, $p=0.45$; power-expansion 0.908 resolvable-only vs. 0.858 locked). Full setup, figures, and numbers in Appendix M.

5.7 Post-hoc secondary analysis: real MemoryAgentBench content

We ran the MemoryAgentBench extension (Section 3.5) rather than only validate its construction, on all strategies including the two ablations

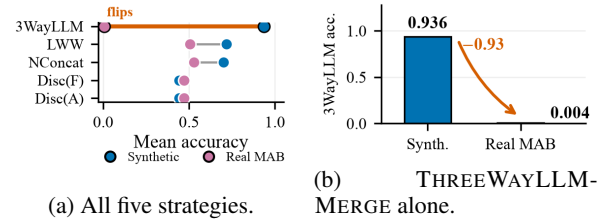


Figure 2: Synthetic vs. real MemoryAgentBench. (a) THREEWAYLLMMERGE goes from best to worst once the signal is removed. (b) Its accuracy drops $0.936 \rightarrow 0.004$ (230-conflict run; per-row spread in Appendix D).

from Section 5.2. MemoryAgentBench’s Conflict_Resolution config has exactly 8 rows total — the *full* real dataset for this task, not a cost-driven subsample. An initial pass capped each row at 15 conflict keys, applied identically to every strategy; to gain more statistical power, we later raised the cap to 50 for every strategy alike, yielding 230 real conflicts ($3.4\times$ the original). Both runs agree in direction and magnitude (Table 7), so the larger sample changed precision, not which strategy looked better. **This result shows that provenance is load-bearing: we treat it as a central finding and a boundary condition on RQ1, not a footnote.**

THREEWAYLLMMERGE scores a mean of 0.004 accuracy on the expanded sample (1 of 230 correct) — *far below* every naive baseline (LWW 0.506, naive concat 0.528, branch-discard 0.472), the *opposite* ranking from the synthetic benchmark. Seven of eight real rows score *exactly* 0, the eighth 0.033 (Appendix D). CONFIDENCERULEMERGE collapses identically, to exactly 0.000, confirming this is a property of the data, not an LLM-specific failure: mab_extension.py never constructs a source-reliability signal (Section 3.5), so every real conflict is an exact confidence tie, which both strategies correctly refuse to break. It committed to a value *zero* times across all 230 conflicts (Section 5.8 gives the full commit/abstention breakdown); naive baselines win only by never abstaining — we did not adjust the extension to flatter it; both the 15- and 50-cap runs are released as-is.

Reading: RQ1’s win is contingent on an explicit, legible reliability signal we engineered into the synthetic benchmark. Absent it, the same calibrated-caution behavior that scored 0.833 on synthetic *ambiguous* conflicts becomes a liability under a scoring rule built for *resolvable* cases — evidence real content may lack the assumed signal,

not poor reasoning. Appendix G gives a condition-by-condition deployment recommendation.

5.8 Abstention-aware evaluation

Raw accuracy scores a calibrated abstention identically to a confused guess: both count as “not correct.” That conflation is exactly what makes Section 5.7’s headline number (0.004) read as a collapse in *capability* rather than a shift in *behavior*. We report commit rate, conditional-on-commit accuracy, wrong-commit rate, and expected utility under three wrong-commit costs (−2, −5, −10, against +1 for a correct commit and −1 for abstaining) — post-hoc, exploratory, computed from already-cached model calls (Table 5, Figure 3; full metric definitions in Appendix K).

Metric	Synthetic ($n=180$)	Real MAB ($n=230$)
Commit rate	0.672	0.000
Cond.-on-commit acc.	0.835	undefined
Wrong-commit rate	0.111	0.000
Flag precision	0.966	—
Flag recall	0.950	—
Expected utility (−2)	+0.011	−1.000
Expected utility (−5)	−0.322	−1.000
Expected utility (−10)	−0.878	−1.000

Table 5: Abstention-aware metrics, THREWAYLLMMERGE. Synthetic pools resolvable+ambiguous ($n=60$ locked run; orthogonal excluded). Post-hoc, exploratory.

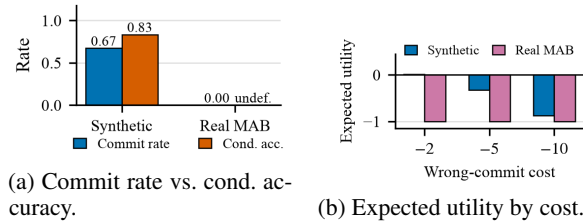


Figure 3: (a) Conditional accuracy is high in both regimes (0.835 synthetic; undefined on MAB, zero commits) — not guessing worse, just committing less. (b) Full abstention (MAB) is a flat −1; synthetic utility falls *below* that line once a wrong commit costs 10× an abstention.

The model’s *policy* is stable across settings: it commits and is right most of the time when a signal exists (synthetic), and abstains completely when it does not (MAB) — consistent behavior, not degraded reasoning. Whether that policy is *good* depends on a cost structure accuracy never asks about: at low wrong-commit cost, committing more beats full abstention; at high cost, it does not. A deployment that cannot specify its own wrong-commit cost cannot know which regime it is in — itself an argument for keeping FLAGGED_UNRESOLVED actionable rather than silently resolved either way.

6 Discussion

Construct validity is not free. Building BRANCHMEM surfaced two piloting failures (and a third in Section J). First, a set-membership bug left every conflict targeting a brand-new key with no ancestor value, so THREWAYLLMMERGE never received the context it depends on — invisible to unit tests with ancestor context already wired in; only auditing transcripts, not aggregate scores, caught it. Second, after the fix, three-way merge still scored near-zero on resolvable conflicts, because ground truth had been a coin flip indistinguishable from “ambiguous.” Tying it to an explicit source-reliability signal (15% noise) is what makes RQ1 a real test.

Timestamps are a trap in the asynchronous case. The MAB-extension construction bugs (Section 3.5) both accidentally correlated a synthetic branch’s timestamp with correctness, rigging LWW’s win or loss (0.00 or 1.00); the fix randomized timestamp ordering independent of correctness: recency is not evidence.

The MAB negative result is the same problem, and what the LLM adds. Section 5.7’s collapse to 0.004 accuracy is the same failure mode on real content: with no exposed reliability signal, “resolvable” is indistinguishable from “ambiguous” — correctly to the model, punitively to our scoring rule. We could fix this in piloting but not on real content, so the synthetic design choice does more work than Table 1 suggests; we did not add a post-hoc rule crediting calibrated abstention, since that is exactly the result-shopping preregistration exists to prevent. The model’s behavior is consistent throughout: it reads resolvable conflicts well when the signal is present and abstains when it is not — only the scoring rule’s reward changes, not the model’s judgment.

7 Conclusion

Three-way, ancestor-aware LLM merging wins decisively given a legible reliability signal, and loses to every naive baseline once that signal is gone. We take this as a boundary condition on RQ1, not a caveat to bury: deployment needs either a genuine reliability signal in memory provenance, or a downstream consumer that treats FLAGGED_UNRESOLVED as actionable, not a failure.

Limitations

Our results come with real caveats, several preregistered, two added after later post-hoc checks.

Model coverage is still narrow. The locked run uses one cheap model (gpt-5.4-nano); the post-hoc gpt-5.4-mini check (Section 5.6) reproduces RQ1’s direction and magnitude, but is a *same-provider* swap only. No non-OpenAI API key is available in this project’s environment, so we could not and did not test any other model family; every LLM-dependent number in this paper (RQ1, RQ3, the ablations, and the RQ2 judge) is OpenAI-only, and whether the reported effects are properties of LLMs reasoning about provenance in general, or specific to this provider’s small/cheap models, is untested and left to future work.

The headline effect is contingent on an engineered signal, demonstrated, not merely hypothesized. Section 5.7’s real-content analysis shows `THREWAYLLMMERGE` scoring *below every naive baseline* (0.004 vs. 0.47–0.53) once the hand-designed source-reliability signal is absent — a measured boundary condition on RQ1, the single most important caveat to this paper’s aggregate numbers.

Sample size and non-determinism. The locked run is $n=60$ scenarios with each LLM call made once, not majority-voted; the pilot flip rate was low (6.7%), but reported numbers carry unquantified variance beyond the paired bootstrap CI. The MAB secondary analysis (Section 5.7) is smaller still (8 rows, capped at 50 conflict keys per row for cost control via `max_conflict_keys`; an initial 15-cap pass, also released, agreed in direction and magnitude) and used only gpt-5.4-nano (not the more capable gpt-5.4-mini checked in Section 5.6); it is purely descriptive, with no significance tests.

No independent validation of ground truth. All ground-truth labels — every “resolvable” case’s correct answer, the 15% source-signal noise parameter, and MemoryAgentBench’s re-derived edit chains — are generated or parsed by construction in code, and no human annotator is available in this project’s environment to check any of them independently. We did not substitute an LLM-based self-consistency check for genuine human validation, since presenting one as the other would mislead. What we did add: an *author self-audit* (not independent, not blind) of 50 randomly sampled MemoryAgentBench parser ex-

tractions against their source text, finding 0 errors (95% Clopper–Pearson CI on the true error rate: $[0, 0.071]$; Section 3.5). This bounds only whether the parser extracts what a context line literally says, not whether “resolvable” ground truth or the noise parameter reflect real-world provenance reliability, which remains completely unvalidated. We provide an audit protocol and a sampled item set (annotation/audit_sample.csv, annotation/audit_instructions.md) to support independent validation. We do not claim independent validation was performed in this submission.

H2’s near-mechanical component. As disclosed in advance, the branch-discard effect is partly an artifact of the benchmark always including orthogonal questions, not pure evidence about real-world information loss.

NLI generalization gap. The NLI detector’s failure reflects one fixed threshold from a modest calibration set, not necessarily an intrinsic limit of NLI-based detection.

Scoring-rule asymmetry. Our scoring rule credits `FLAGGED_UNRESOLVED` on the synthetic ambiguous category but treats it as wrong on MemoryAgentBench’s resolvable category — an inconsistency in our evaluation design, not just the two content sources; a consistent abstention-crediting rule is future work.

Ethical Considerations

BRANCHMEM is a methods-and-benchmark paper. All scenarios in the locked run are synthetically generated from a fixed template pool and contain no personal, sensitive, or human-subject data; no human annotation was collected. The MemoryAgentBench extension reuses a publicly released, MIT-licensed research dataset within its intended research use. The only external service used is a commercial LLM API, accessed under its terms; all calls are cached and committed for auditability, and the run is inexpensive by design. We foresee no direct risk of harm from this work; the broader technique—automatically merging an agent’s memory—should, in deployment, retain the human-in-the-loop FLAG behavior our benchmark rewards, so that genuinely ambiguous conflicts are surfaced rather than silently resolved.

References

Ran El-Yaniv and Yair Wiener. 2010. On the foundations of noise-free selective classification. *Journal*

of Machine Learning Research, 11:1605–1641.

Clarence A. Ellis and Simon J. Gibbs. 1989. [Concurrency control in groupware systems](#). In *Proceedings of the 1989 ACM SIGMOD International Conference on Management of Data*, pages 399–407.

Yonatan Geifman and Ran El-Yaniv. 2017. Selective classification for deep neural networks. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 30.

Yuanzhe Hu, Yu Wang, and Julian McAuley. 2025. [Evaluating memory in LLM agents via incremental multi-turn interactions](#). *arXiv preprint arXiv:2507.05257*. To appear at ICLR 2026.

Sajjad Khan. 2026. [S-Bus: Automatic read-set reconstruction for multi-agent LLM state coordination](#). *arXiv preprint arXiv:2605.17076*.

Martin Kleppmann, Adam Wiggins, Peter van Harenberg, and Mark McGranaghan. 2019. [Local-first software: You own your data, in spite of the cloud](#). In *Proceedings of the 2019 ACM SIGPLAN International Symposium on New Ideas, New Paradigms, and Reflections on Programming and Software (Onward!)*, pages 154–178.

Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. 2020. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 33, pages 9459–9474.

Shayne Longpre, Kartik Perisetla, Anthony Chen, Nikhil Ramesh, Chris DuBois, and Sameer Singh. 2021. [Entity-based knowledge conflicts in question answering](#). In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 7052–7063. Association for Computational Linguistics.

Charles Packer, Vivian Fang, Shishir G. Patil, Kevin Lin, Sarah Wooders, and Joseph E. Gonzalez. 2023. [MemGPT: Towards LLMs as operating systems](#). *arXiv preprint arXiv:2310.08560*.

Vikas Reddy and Sumanth Challaram. 2026. [Don’t ask the LLM to track freshness: A deterministic recipe for memory conflict resolution](#). *arXiv preprint arXiv:2606.01435*.

Nils Reimers and Iryna Gurevych. 2019. [Sentence-BERT: Sentence embeddings using Siamese BERT-networks](#). *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 3982–3992.

Dipankar Sarkar. 2026. [Before the pull request: Mining multi-agent coordination](#). *arXiv preprint arXiv:2606.19616*.

Marc Shapiro, Nuno Preguiça, Carlos Baquero, and Marek Zawirski. 2011. [Conflict-free replicated data types](#). In *Proceedings of the 13th International Symposium on Stabilization, Safety, and Security of Distributed Systems (SSS)*, pages 386–400. Springer.

Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. 2023. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 36.

A ThreeWayLLMMerge Prompt

The system prompt instructs the model that it is reconciling two independently updated branches of an agent’s memory that diverged from a common ancestor while disconnected. For each conflicting key it is given the ancestor value (or null), and each branch’s current value, source (user/observation/inference), and confidence. It is told to resolve using whatever signal actually distinguishes the two updates—e.g. a user-sourced, higher-confidence update is ordinarily more trustworthy than an inferred, lower-confidence one, and the ancestor value indicates which branch’s update is a genuine correction versus a stale duplicate—and *not* to use timestamps, since disconnected branches have no shared clock. If source, confidence, and ancestor context give no real basis to prefer one branch, it must flag the conflict unresolvable rather than guess. The model returns strict JSON: for each key, one of {kept_from_a, kept_from_b, merged, flagged_unresolved}, a reconciled value (or null), and a one-sentence justification. Non-conflicting keys are resolved in code and never sent to the model. Below is the literal text, verbatim from branchmem/merge/three_way_llm.py (constants _SYSTEM_PROMPT and _PROMPT_TEMPLATE), so readers can judge instructions such as the timestamp prohibition for themselves rather than trust our paraphrase.

System prompt (verbatim).

You are reconciling two independently-updated branches of an agent’s memory that diverged from a common ancestor while disconnected from each other. For each conflicting (entity, predicate) key, you are given the common ancestor’s value (if any) and each branch’s current value, source, and confidence. ‘source’ is how that update was recorded: ‘user’ means the user explicitly stated or corrected it; ‘inference’ means the agent inferred or guessed it; ‘observation’ means the agent observed it indirectly. Resolve each conflict using whatever signal actually distinguishes the two updates -- e.g. a user-sourced, higher-

confidence update is ordinarily more trustworthy than an inferred, lower-confidence one; the ancestor value can indicate which branch's update is more likely a genuine correction vs. a stale duplicate. Do NOT use timestamps to decide, even if you can infer relative recency -- two branches that were disconnected from each other have no shared clock, so 'later' does not mean 'more correct' here. If source, confidence, and ancestor context give no real basis to prefer one branch over the other (e.g. both are equally plausible, equally-sourced preference changes), explicitly flag it as unresolvable rather than guessing.

User prompt template (verbatim, placeholders filled per merge call).

```
Common ancestor value for each key (null if the
key didn't exist yet):
{ancestor_json}

Branch A's current value/source/confidence for
each conflicting key:
{branch_a_json}

Branch B's current value/source/confidence for
each conflicting key:
{branch_b_json}

For EACH key listed above, decide how to
reconcile it. Respond with strict JSON only,
no other text, in this exact shape:

{"resolutions": [
  {"entity": "...", "predicate": "...", "
  resolution": "kept_from_a" | "kept_from_b" |
  "merged" | "flagged_unresolved",
  "value": "<the reconciled value, or null if
  flagged_unresolved>", "justification": "<one
  sentence>"}
]}
```

{ancestor_json}, {branch_a_json}, and {branch_b_json} are substituted at call time with the actual per-key JSON blocks for that merge's conflicting keys only; non-conflicting keys are never sent to the model. Appendix B shows one such call filled in, with the model's real response.

B Worked Example

To make the mechanism concrete rather than only described in the abstract, this is one real, representative transcript from the committed model-call cache (llm_cache/, content-hash addressed and described in Section 5's Reproducibility paragraph), reproduced verbatim. The individual cache entry does not retain a back-link to a specific locked-run scenario_id, so we present it as illustrative of the mechanism rather than tied to one row of Table 2; its pattern (source/confidence resolves two keys, symmetric equal-source conflict

gets flagged) matches the aggregate category behavior reported there.

Inputs sent to the model. One entity (erin) with three conflicting keys. The ancestor only had a value for one of them (opinion_of_new_policy = "neutral"); the other two are new keys each branch introduced independently, so the ancestor value is absent for those.

Key	Branch A	Branch B	Ancestor
manager_name	riley chen user, 0.95	taylor reyes inference, 0.55	—
opinion_of_new_policy	skeptical observation, 0.7	supportive observation, 0.7	neutral
phone_number	555-0142 inference, 0.55	555-0101 user, 0.95	—

Model's actual response (verbatim justification fields).

- **manager_name** → KEPT_FROM_A, "riley chen": *"Branch A is user-sourced with higher confidence (0.95) than Branch B's inference (0.55)."*
- **opinion_of_new_policy** → FLAGGED_UNRESOLVED: *"Both branches are observation-sourced with equal confidence (0.7) and the ancestor value ('neutral') does not distinguish which is correct."*
- **phone_number** → KEPT_FROM_B, "555-0101": *"Branch B is user-sourced with higher confidence (0.95) than Branch A's inference (0.55)."*

All three justifications cite exactly the signal the system prompt says to use (source and confidence) and nothing else; none mentions recency, even though the model is not shown timestamps at all for these keys. The two resolvable conflicts are resolved toward whichever branch is user-sourced and higher-confidence, regardless of whether that is Branch A or Branch B — the model is not simply defaulting to one branch. The third key is a textbook ambiguous case in our taxonomy (Section 3.4): identical source, identical confidence, uninformative ancestor — and the model flags it instead of guessing, exactly the behavior Section 5.7 shows becoming costly once the scoring rule assumes a signal is present.

C Per-Divergence-Span Results

Section 5 reports that divergence duration had little effect on THREWAYLLMMERGE (0.914/0.957/0.936 at spans 4.0/10.0/20.0). Table 6 gives the same breakdown for all five strategies, computed from the locked run's per-scenario

Strategy	span 4.0	span 10.0	span 20.0
last_writer_wins	0.700	0.736	0.714
naive_concat	0.707	0.700	0.693
branch_discard (always_b)	0.436	0.443	0.450
branch_discard (fewer_upd.)	0.436	0.443	0.450
three_way_llm	0.914	0.957	0.936

Table 6: Accuracy by divergence span and strategy (20 scenarios/span). Descriptive; span was not a preregistered factor.

Row	<i>n</i> pairs	<i>n</i> questions	3-way acc.
0	29	29	0.000
1	23	23	0.000
2	27	27	0.000
3	30	30	0.033
4	29	29	0.000
5	26	26	0.000
6	35	35	0.000
7	31	31	0.000

Table 7: Per-row detail behind Section 5.7’s 0.004 mean (max_conflict_keys=50). Other strategies (CONFIDENCERULEMERGE scores 0.000 throughout) are in the released JSON. An earlier pass (15-cap, 67 conflicts, mean 0.056) agrees in direction and magnitude.

records in results/final/results.json (20 scenarios per span, 60 total). No strategy’s ranking changes across spans, and THREEWAYLLMMERGE’s own accuracy varies by only 0.043 across a $5\times$ range of divergence duration — consistent with the aggregate result not being an artifact of a particular span.

D MemoryAgentBench Per-Row Detail

Section 5.7’s headline number (0.004 mean accuracy for THREEWAYLLMMERGE) is a mean over only 8 real Conflict_Resolution rows — the full dataset, not a subsample. Table 7 gives the individual rows from the expanded run (mab_secondary_analysis_expanded.json, max_conflict_keys=50, 230 total conflicts), to make the spread behind that mean transparent rather than reporting only the aggregate.

Seven of eight rows score exactly 0; row 3 is the only exception, at 0.033 (1 of 30 questions). This is not one degenerate row dragging down an otherwise-healthy mean — the near-uniform floor is what we would expect if the mechanism identified in Section 5.7 (no source-reliability signal to read, so the model flags almost every conflict) operates consistently across rows. Splitting by commit/flag makes this precise: across all 230 conflicts, THREEWAYLLMMERGE committed to an answer zero times — every single response was FLAGGED_UNRESOLVED. Its conditional-on-commit accuracy is therefore undefined, not

Threshold band	Embedding F1	NLI F1
0.10–0.25	—	0.938
0.30–0.55	—	0.929
0.60–0.75	—	0.919
0.80–0.90	0.938	0.909

Table 8: Pilot-set F1 by threshold band (embedding swept [0.30, 0.90], NLI [0.10, 0.90], step 0.05). Embedding flat at 0.938 from 0.80 up; NLI peaked at 0.938 for 0.10–0.25, declining above. Locked: embedding 0.80, NLI 0.20 — plateau interiors chosen for robustness, not the single best pilot value.

merely low; there were no commits to be accurate or inaccurate about. This is stronger and more mechanically legible evidence than the $n=8$ -row aggregate alone would suggest, since it does not depend on between-row averaging at all. We did not measure per-row parser coverage separately; only the aggregate figure reported in Section 3.5 (99.9% over 18,337 unique context lines, across the full dataset the 8 rows are drawn from) was logged.

E Detector Threshold Calibration Sweep

Section 5 states embedding and NLI thresholds were chosen from a plateau rather than an edge value. Table 8 gives the sweep behind that claim, against the 60 ground-truth conflict pairs in the held-out pilot set (ANALYSIS_PLAN.md §3). Precision was 1.0 at every threshold tested for both detectors — across the entire sweep, neither detector ever produced a false positive; all sweep error is missed conflicts (false negatives).

Judge F1 drift, bootstrapped. On the pilot calibration set, both cheap detectors tied at F1 0.938 against an uncalibrated-default judge F1 of 0.966 (gaps ≈ 0.028 , both inside the 0.05 tolerance); on the independently-generated, three-times-larger locked set, the judge’s own F1 rose to 0.986. Bootstrapped 95% CIs (2000 resamples) show why this need not be systematic: judge F1 0.966 [0.929, 0.992] (pilot) vs. 0.986 [0.971, 0.997] (locked) overlap substantially, consistent with ordinary sampling variation at $3\times$ the sample size. The gap itself bootstraps to 0.029 [0.006, 0.056] (embedding) and 0.061 [0.036, 0.089] (NLI).

F Locked Scenario Configuration

The locked run used: 1 entity, 4 ancestor facts, 2 orthogonal additions per branch, 2 resolvable conflicts, 1 ambiguous conflict, timestamp-correctness correlation = 0.5, source-signal noise = 0.15, over three divergence spans

Condition	Recommended behavior
Reliable provenance available (distinguishable source/confidence)	Three-way merge can resolve; expect accuracy near the synthetic resolvable-conflict result (0.858).
No provenance, or equal confidence on both branches	Do not force a resolution; flag. Committing anyway is indistinguishable from guessing (Section 5.7).
Downstream consumer can act on an unresolved flag	Treat the flag as a successful outcome, not a failure — it is the correct output when no signal exists.
Downstream consumer requires a forced single answer	A naive baseline (LWW, concat) may score higher under accuracy-only metrics, but this reflects the metric, not increased safety — see Section 6’s scoring-rule asymmetry.

Table 9: Deployment guidance implied by RQ1’s boundary condition.

{4.0, 10.0, 20.0} (20 scenarios each, $n=60$ total, 180 conflict pairs: 120 resolvable, 60 ambiguous), seed 2026. Detector thresholds: embedding 0.80, NLI 0.20, calibrated on the held-out pilot set.

Documentation correction (found during this revision). ANALYSIS_PLAN.md §1 documented one orthogonal addition per branch; re-auditing scripts/run_experiment.py found it never overrides ScenarioConfig’s orthogonal-count defaults, so two per branch actually ran (confirmed empirically: results/final/results.json’s per-scenario question count is $7=2+2+2+1$, not $5=1+1+2+1$). This is a documentation error, not a results change — every accuracy number in this paper is computed correctly from what actually ran; only the earlier prose description of the config (here and in ANALYSIS_PLAN.md) was wrong. It does not affect any conclusion: the orthogonal category is tied at 1.000 across every strategy regardless of whether 2 or 4 orthogonal keys are involved.

G Deployment Decision Matrix

Table 9 translates Section 5.7’s boundary condition into condition-by-condition guidance for a system builder deciding how to consume THREEWAYLLMMERGE’s output.

H Resolvable-Category Power Expansion Detail

Full per-strategy numbers for the post-hoc power-expansion replication (Section 5.6): base seed

Strategy	New sample	Locked run
THREEWAYLLMMERGE	0.908	0.858
CONFIDENCERULEMERGE	0.917	—
TWOWAYLLMMERGE	0.917	—
RAWTEXTLLMMERGE	0.000	—
last_writer_wins	0.483	0.508
naive_concat	0.417	0.450
branch_discard (both)	0.583	0.550

Table 10: Resolvable-only accuracy, new $n=120$ sample vs. locked $n=120$. On the new sample, CONFIDENCERULEMERGE and TWOWAYLLMMERGE both tie THREEWAYLLMMERGE ($p=0.317$ both, diff -0.008); RAWTEXTLLMMERGE again scores 0.000. All four naive-baseline differences vs. THREEWAYLLMMERGE are large and significant (Wilcoxon $p < 10^{-5}$; e.g. vs. last_writer_wins: diff $+0.425$, 95% CI $[0.325, 0.525]$).

2029, 120 new resolvable pairs, kept disjoint from and not pooled with the locked ($n=60$, seed 2026) run. Dashes mark strategies for which the locked run’s per-scenario logs recorded only overall (not per-category) accuracy, so a resolvable-only locked-run figure is not available for direct comparison; the new sample’s own significance tests (below the table) are self-contained and do not rely on that comparison.

I Balanced Detector Benchmark Detail

Section 5.4’s locked set has no negative examples by construction, so precision is mechanically 1.0 for every detector — it measures recall only. We built a balanced, hand-authored 48-pair set across six categories: genuine contradiction and subtle ambiguous conflict as true positives ($n=16$), and paraphrase, entailment, unrelated-same-entity content, and preference refinement as true negatives ($n=32$), then re-ran all three detectors (post-hoc, exploratory; does not change or retract RQ2’s locked result). Full metrics below.

Detector	Prec.	Rec.	F1	Spec.	AUROC
embedding ($t=0.80$)	0.381	1.000	0.552	0.188	0.463
NLI ($t=0.20$)	0.667	1.000	0.800	0.750	0.871
LLM judge	0.722	0.813	0.765	0.844	0.828 [†]

Table 11: Balanced detector benchmark (48 pairs, 16 conflict / 32 non-conflict). [†]LLM judge emits a binary score only, so its AUROC collapses to a single operating point rather than a true ranked curve; reported for completeness, not read as equivalent to the continuous-score detectors’ AUROC.

Per-category accuracy: embedding scores 1.0 on contradiction and ambiguous-conflict but only 0.125 on paraphrase and entailment, 0.5 on preference-refinement, and 0.0 on unrelated-same-entity (it measures surface dissimilarity, not semantic incompatibility, so it flags most non-

conflicts as conflicts). NLI scores 1.0 on every category except unrelated-same-entity (0.0). The LLM judge scores 0.875 on contradiction, 1.0 on paraphrase and preference-refinement, 0.875 on entailment, 0.75 on ambiguous-conflict, and 0.5 on unrelated-same-entity — the most even profile of the three, but no detector handles unrelated content sharing an entity and predicate well, since none of the three signals was built to distinguish “irrelevant” from “incompatible.”

J Semantic-Resolvable Category Detail

Full construction and transcript detail for Section 5.5’s semantic_resolvable category ($n=150$, seed 3001, disjoint from the locked run’s 2026): both branches get identical source (observation) and confidence (0.7), and independently-randomized timestamps, so no structured field distinguishes them.

A construct-validity bug we caught and fixed.

The first version of this experiment stored the ancestor’s constraint under a *different* predicate than the conflicting key (e.g. dietary_preference vs. dinner_choice). THREEWAYLLMMERGE’s prompt includes the ancestor’s value only for the exact colliding key (Section 3.1), so the constraint was invisible to the model regardless of its reasoning ability — a bug in our harness, not a finding about semantic reasoning, in the same family as Section 6’s piloting bugs. We caught it by inspecting a cached response directly rather than trusting the aggregate 0.000, rewrote the templates so the constraint lives at the conflicting key, and re-ran (new API calls; both the flawed and corrected runs are logged in PROGRESS.md).

Result after the fix. Every confidence-independent strategy still scores *exactly* 0.000: CONFIDENCERULEMERGE correctly flags the equal-confidence tie (as designed); TWOWAYLLMMERGE and RAWTEXTLLM-MERGE structurally lack the ancestor content needed here; and — the more informative result — THREEWAYLLMMERGE, which *does* receive the constraint, also scores 0. Inspecting individual transcripts shows inconsistent, not absent, behavior: some conflicts are correctly flagged; some are merged into an incoherent combination of both values (e.g. “wants chicken for dinner and wants tofu for dinner”); and at least one is a confident *wrong* commit that ignores a stated allergy constraint entirely. This sharpens rather than undercuts RQ1’s reframing (Section 5.2): the

LLM’s aggregate advantage over naive baselines is attributable to reading a structured confidence field, and we found no category in this benchmark, semantic or otherwise, where its raw-text reasoning reliably exceeds that.

K Abstention Metric Definitions

Metrics reported in Section 5.8 (Table 5), computed by branchmem/eval/abstention_metrics.py over pooled resolvable+ambiguous downstream questions (orthogonal excluded — there is nothing to abstain from). Let N be the number of in-scope questions, C the number committed (resolution $\neq$ FLAGGED_UNRESOLVED), C_{correct} the number of those that are correct, and $F = N - C$ the number flagged.

- **Commit rate** = C/N . **Abstention rate** = F/N .
- **Conditional-on-commit accuracy** = C_{correct}/C (undefined, not zero, when $C=0$).
- **Wrong-commit rate** = $(C - C_{\text{correct}})/N$.
- **Flag precision** = (flags on genuinely ambiguous questions) / F . **Flag recall** = (ambiguous questions flagged) / (total ambiguous questions) — equivalent to the ambiguous-category accuracy in Table 2.
- **Expected utility** (per question, averaged over N): a correct commit scores +1, an abstention scores −1, and a wrong commit scores the regime’s cost (−2, −5, or −10 in Table 5).
- **Coverage** = commit rate. **Risk** = 1 − conditional-on-commit accuracy (undefined when coverage is 0), following the risk-coverage framing of selective classification (El-Yaniv and Wiener, 2010; Geifman and El-Yaniv, 2017).

All metrics here are post-hoc and exploratory (ANALYSIS_PLAN_ADDENDUM.md, item A1); none are part of ANALYSIS_PLAN.md’s confirmatory tests.

L Extended Related Work

Full discussion of four related-work points introduced briefly in Section 2, moved here since each is either already distinguished from our setting in Section 1, or is background supporting a single RQ rather than the paper’s central boundary- condition claim.

L.1 Agent memory and memory benchmarks

Reflexion (Shinn et al., 2023) and MemGPT (Packer et al., 2023) give single agents persistent, self-managed memory — verbal self-reflection and OS-inspired paging — but assume one continuously-running agent, not two independently-updated copies to reconcile. MemoryAgentBench (Hu et al., 2025) benchmarks retrieval, test-time learning, long-range understanding, and selective forgetting for such single-timeline agents; its Fact Consolidation task is our extension’s raw material (Section 3.5).

L.2 Live multi-agent coordination vs. offline branch reconciliation

S-Bus (Khan, 2026) targets concurrent LLM agents sharing mutable state over a common substrate: it reconstructs each agent’s read set and uses optimistic concurrency control to abort or retry conflicting commits — a *prevention/abort* model for agents online together. Our branches already committed divergent histories offline, so abort is not an option; BRANCHDISCARD is the natural post-hoc analogue, testing whether that cheap move is good enough (RQ3).

L.3 Single-timeline freshness and temporal conflict resolution

Reddy and Challaram (2026) show that for a single-timeline list where a counterfactual edit gets a higher serial, a deterministic “pick the maximum serial” rule beats LLM judgement. Our problem lacks such an order: asynchronous branches carry independent local clocks, so recency is uninformative by construction (Section 3.4).

L.4 Conflict detection, NLI, and semantic similarity

Prior work studies conflicts between an LLM’s parametric knowledge and retrieved context (Longpre et al., 2021), and retrieval-augmented generation (Lewis et al., 2020) — model memory vs. an external source at inference time. We instead study conflict between two versions of an agent’s *own* memory that diverged offline. Our cheap detectors (RQ2) build on sentence embeddings (Reimers and Gurevych, 2019) and NLI-style contradiction scoring; Section 5.4’s balanced benchmark tests both against non-conflict categories this line of work is designed to not flag.

M Post-hoc Robustness Checks Detail

Full setup and figures for Section 5.6’s two post-hoc robustness checks.

M.1 A more capable model

We ran gpt-5.4-mini in place of gpt-5.4-nano for THREEWAYLLMMERGE on the same 20 scenarios (divergence span 4.0), everything else identical. Figure 4 shows the result is reassuring but modest: mean accuracy 0.929 (mini) vs. 0.914 (nano), paired difference +0.014, 95% CI $[-0.014, 0.043]$, Wilcoxon $p = 0.45$ — not distinguishable from noise. The direction and magnitude of RQ1 hold with a more capable model, and a *same-provider* check only — we did not have access to a different model family for this study.

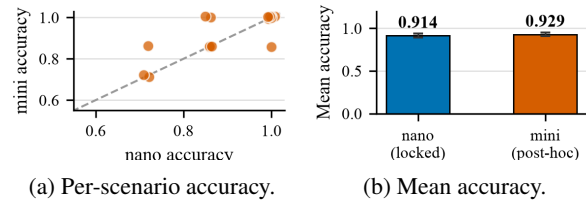


Figure 4: gpt-5.4-mini vs. locked gpt-5.4-nano, same 20 scenarios. Paired diff. +0.014, 95% CI $[-0.014, 0.043]$, $p = 0.45$: small, not significant.

M.2 A resolvable-category replication

Since the resolvable category carries essentially all of RQ1’s non-mechanical evidence, we generated a second, disjoint-seed sample (120 new resolvable pairs, base seed 2029) for more statistical power. Post-hoc, kept separate from Tables 1–2: it replicates closely (0.908 resolvable-only vs. 0.858 locked, all four naive-baseline differences remain large and significant alone, Wilcoxon $p < 10^{-5}$; CONFIDENCERULEMERGE/TWOWAYLLMMERGE again land within noise of it, $p = 0.32$ both; RAWTEXTLLMMERGE again scores exactly 0.000) — 240 independently-drawn resolvable pairs total, every qualitative conclusion unchanged. Full numbers in Table 10, Appendix H.